Transformer 模型

第 2 章（上）

第 2 课时 本课时讲解 2.1–2.2、2.3（前半）

注意力机制 · 模型结构 · 训练与推理 · 代码实践

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录 第 2 课时

2.1 Transformer 概述
自注意力机制与 Transformer 的核心价值 第 3 页

2.2 注意力机制
序列表征、注意力概念与注意力分数 第 7 页

2.3 Transformer 模型结构
输入层、编码器、解码器与输出层 第 24 页

2.4 Transformer 训练和推理
并行训练与自回归推理 第 76 页

2.5 关键代码实践
缩放点积注意力与多头注意力的 PyTorch 实现 第 85 页

2.6 小结
本章要点与位置编码的必要性（附录 2.6.1） 第 92 页

本课时内容 2.1 Transformer 概述、2.2 注意力机制、2.3 Transformer 模型结构（前半）（其余部分见第 3 课时）

2.1

Transformer 概述

Transformer 以自注意力取代顺序依赖，成为深度学习与大语言模型的基石架构。

本节内容

核心价值

影响范围

自注意力机制

Transformer：深度学习的基石

自 2017 年提出以来，Transformer 已成为深度学习领域的基石性突破和主导力量，重塑了自然语言处理、计算机视觉、语音识别与多模态学习的研究格局。

核心价值 1

强大的表征学习

自注意力直接建模序列中元素之间的关系，学到的表征能够刻画丰富的语义信息。

核心价值 2

架构的统一性

同一架构可以应用于不同模态，文本、图像等数据都能用同样的结构处理。

核心价值 3

卓越的可扩展性

模型规模增大时性能持续提升，因此可以随规模不断扩展。

> **注** 它不仅是大语言模型和多模态大模型的绝对核心，也是推动通用人工智能发展不可或缺的关键技术。

影响：从语言到视觉与多模态

Transformer 先革新了自然语言处理，再跨界进入视觉，并成为多模态模型的统一桥梁。

自然语言处理

BERT：双向编码器

通过双向编码器革新了语言理解。

自然语言处理

GPT 系列：自回归解码器

以自回归解码器和大规模训练，展示了惊人的文本生成与推理能力。

计算机视觉

ViT：视觉新主干

证明纯 Transformer 可以超越 CNN，成为视觉领域的新主干。

多模态

CLIP 与 DALL-E

CLIP 实现图文对齐；DALL-E 根据文本描述生成图像。

文献 Devlin et al., BERT；Radford et al., GPT；Dosovitskiy et al., ViT；Radford et al., CLIP；Ramesh et al., DALL-E

自注意力：模仿人类的注意力集中

注意力的核心优势：针对特定任务，自动学习各元素的重要性并分配相应权重。

类比：高速公路驾驶

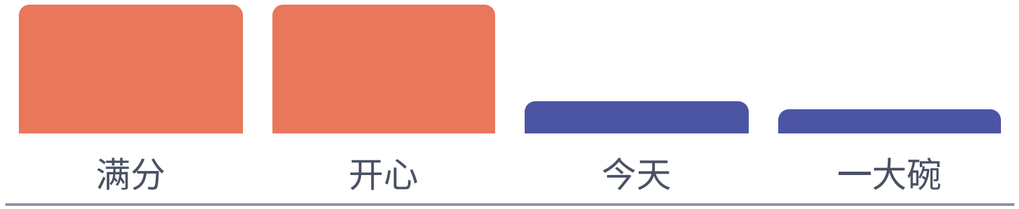
聚焦与安全行驶相关的元素

- 视野中有大量信息：路旁树木、远处风景、广告牌。
- 聚焦前方及侧方车辆、车道标线与交通信号等动态元素。
- 主动降低对路旁静态景物的注意力。

例子：情感分析

关键词获得更高的关注度

“小明今天考试得了满分，回家后开心地吃了一大碗冰淇淋”



柱高为示意，表示相对关注度。

2.2

注意力机制

Transformer 的核心是让每个词元直接读取序列中的其他词元，而不是把历史全部压缩进一个隐状态。

本节内容

2.2.1 序列表征
2.2.2 注意力概念
2.2.3 注意力分数

序列表征：把词序列组合成文本表征

表征是数据在模型中的数学表达，文本表征的目标是把非结构化的自然语言转化为结构化的数值向量。文本本质上是离散的词序列，需要序列建模技术将词的表征组合起来。

直接平均

把序列中所有词的表征取平均作为文本表征。能够全局反映文本语义，但忽略了各词对语义贡献的差异。

真正需要的能力

高效组合序列信息，并区分不同词的重要性。这正是注意力机制要解决的问题。

注：Word2Vec（见 1.4.4 节）只是对词的表征学习，并非对序列表征学习。

词的静态向量、上下文词向量与序列向量

同一个“苹果”，在词表里只有一个向量；放进句子里，每个位置各自得到一个随上下文变化的向量；整个句子还能再压成一个向量。三个层次的形状与含义都不同。

1 · 词的静态向量　Word2Vec

$1 \times d$

从词表查表得到，一个词一个向量，与它出现在哪里无关。词形相同，向量就相同。

2 · 词在序列中的向量

$n \times d$ **的某一行**

经注意力加权融合后，每个位置各自得到一个向量。同一个词出现在不同位置，向量不同。

3 · 整个序列的向量

$1 \times d$

把 n 个位置的向量做池化或取末位，整句压成一个向量，用于分类、检索与句相似度。

层次	形状	随上下文变化	“苹果”在两句话里	来源
词的静态向量	$1 \times d$	否	完全相同	Word2Vec 查表（1.4.4 节）
词在序列中的向量	$n \times d$ 的一行	是	“吃苹果”偏水果，“苹果手机”偏品牌	注意力层输出 $\boldsymbol{Z}$
整个序列的向量	$1 \times d$	—	两句句义不同，句向量也不同	池化，或取末位词元

例　“他吃了一个苹果。”与“苹果发布了新手机。”：静态词向量无法区分两个“苹果”的差别，上下文词向量可以，两句话的序列向量也不同。

从压缩记忆到按需读取

循环网络把历史压缩进隐状态；注意力保留序列中每个时刻的表征，需要时按权重直接读取。

循环神经网络

压缩记忆：逐步整合历史

$\boldsymbol{x}_1$ 词元 → $\boldsymbol{h}_1$ 隐状态 → $\boldsymbol{h}_n$ 固定维度

历史被压缩进固定维度的隐状态，细节难以保留。

注意力机制

按需读取：保留每个表征

$\boldsymbol{x}_1, \cdots, \boldsymbol{x}_n$ 原始表征全部保留 → **权重** 动态计算相关性

按权重对历史各时刻的表征加权组合。

例 读第 9 章时需要参考第 1 章：可依靠大脑的压缩记忆（类似 RNN），也可直接翻到第 1 章定位（类似注意力）。注意力的检索避免了信息遗失，代价是存储与计算复杂度上升。

从文本到词元：查表得到编号序列

文本
原始字符串

→

词元
分词后的基本离散单元

→

词元编号
词表中的唯一编号

→

词向量
词嵌入表示（见 1.4.3 节）

- 词元是文本划分后的基本离散单元；字符级分词器中，每个字符即一个词元（见 1.4.2 节）。
- 独立的词向量只能编码词元本身的基础语义；理解词元在序列中的实际语义，必须考虑上下文。
- 传统的规则方法难以覆盖复杂多变的语言模式。注意力机制以可学习、数据驱动的方式，直接建模并量化任意两个词元之间的关联程度。

中文例子：主谓宾关系跨越了修饰语

“喜欢深度学习的学生提交了一份精彩的报告”——句子骨架只有“学生—提交—报告”三块，其余都是修饰语；而修饰语里的“喜欢”管的是“深度学习”，不是“学生”。

词元	词性	句法角色	与谁相关
学生	名词	主语	提交（主谓），相隔 3 个词元
提交	动词	谓语	学生（主谓）、报告（动宾）
报告	名词	宾语	提交（动宾）
喜欢	动词	定语从句谓语	深度学习（动宾），不是“学生”
精彩	形容词	定语	报告（定中）
的	助词	结构助词	标记“的”字结构，本身不承载语义

表中每一行的“与谁相关”就是一条需要建模的边；有些边相邻，有些隔了三四个词元。

> **关键** “学生”要读到“提交”，跨度 3 个词元；“喜欢”要读到“深度学习”而不是紧邻的“学生”。这样的边在句子里有成百上千条，无法逐条写规则，只能交给可学习的权重。

传统做法的链条

一串模型，逐级传误差

- 分词：中文没有空格，先要切词，再处理歧义与未登录词。
- 词性标注：同形词常有多类，需要 HMM、CRF 一类模型。
- 句法分析：依存或短语结构语法，规则写不完，还要用转移式、图式模型训练。
- 前一步错了，后一步全错；换一种语言，整条链要重做。

英文例子：倒装与定语从句

“Never have I seen a model that captures such long-range dependencies so well.”——否定副词提前引起倒装，从句谓语又要与先行词一致：两处关系都不在相邻词元之间。

现象	关系	相隔	说明
倒装	have → I	相邻	Never 提到句首，助动词 have 移到主语 I 之前，主谓一致按 I 判断。
倒装	seen → I	1 个词元	谓语 seen 的逻辑主语是 I，中间隔着助动词。
定语从句	that → model	相邻	关系代词 that 指代先行词 model，而不是它前面的冠词 a。
定语从句	captures → model	4 个词元	从句谓语与先行词 model 单数一致，不能看紧邻的复数 dependencies。
动宾	captures → dependencies	5 个词元	从句谓语与宾语之间隔着定语 long-range。

传统做法

语法知识与工具各管一段

先要有一套短语结构或依存语法标注，再训练转移式、图式解析器；主谓一致、从句边界这类现象还要额外写规则与特征。

注意力

直接学“谁与谁相关”

不必先切分句子成分：每个位置对全部位置算一行分数，权重大的位置就是它读取的对象。语法关系成为可学习的权重。

结论 语言的结构性体现在词与词的关系上。把“关系”做成可学习的量，就不必为每种语言、每种现象各写一套分析器。

词元之间的注意力关系（示例 1）

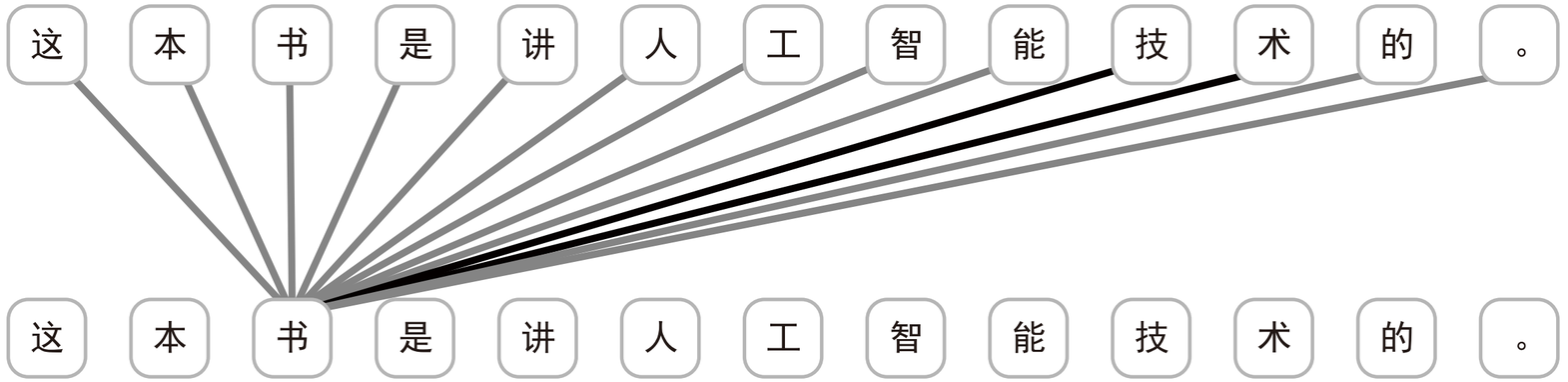


图 2.1　词元在序列中的注意力关系（示例 1）

- 连线越粗，表示两个词元之间的注意力权重越大。
- 句子“这本书是讲人工智能技术的。”中，词元“书”与内容词元“技术”可能有较强关联，与句末标点“。”的关联则相对较弱（并非完全无关）。

图源：《大语言模型：训练、精调与对齐》图 2.1

多头视角：同一句子的不同关系



图 2.2　词元在序列中的注意力关系（示例 2）



图 2.3　词元在序列中的注意力关系（示例 3）

多头注意力让模型从不同“角度”关注词元间的不同关系，更有效地捕捉文本内部复杂的语义关联。

图 2.2：词元“这”的注意力权重高度集中在指代对象“书”上。未显示的连线不代表绝对无关，仅表示关联度低于可视化阈值。

图 2.3：对于词元“书”，一个注意力头聚焦于与谓语动词“是”的语法关系（强关联），同时关注与宾语“技术”的语义联系。

图源：《大语言模型：训练、精调与对齐》图 2.2、图 2.3

注意力的作用：上下文感知的表示

学习权重
词元之间的关联程度

→

加权融合
对词向量加权融合

→

动态表示
每个词元的上下文感知语义

→

序列表示
序列级别的语义表示

- 通过调整注意力权重，可以灵活改变每个词元的语义表示，聚焦于对当前任务更重要的上下文信息。
- 注意力的核心是系统性地学习序列中词元之间的关联程度（注意力权重），并利用它们对词向量加权融合。
- 具体的注意力计算（缩放点积注意力）详见 2.3.4 节。

注意力分数：点积度量词元相关性

注意力机制需要度量词元之间的相关性，常用方法之一是计算词向量之间的点积，作为注意力分数。

$$\boldsymbol{x}_i \in \mathbb{R}^d$$

$$\boldsymbol{X} = [\boldsymbol{x}_1; \boldsymbol{x}_2; \cdots; \boldsymbol{x}_n] \in \mathbb{R}^{n \times d}$$

$\boldsymbol{x}_i$ 为第 i 个词元的词向量，d 为向量维度；n 个词元的词向量按行拼接，得到词矩阵 $\boldsymbol{X}$。

1 · 词与词

$\boldsymbol{s}_{ij}$

两个词元 i、j 之间的标量分数，见图 2.4(a)。

2 · 词与序列

$\boldsymbol{S}_i$

一个词元与全部 n 个词元的分数行向量，见图 2.4(b)。

3 · 序列与序列

$\boldsymbol{S}$

所有词元对之间的 $n \times n$ 自注意力分数矩阵，见图 2.4(c)。

词元之间的注意力分数 s_{ij}

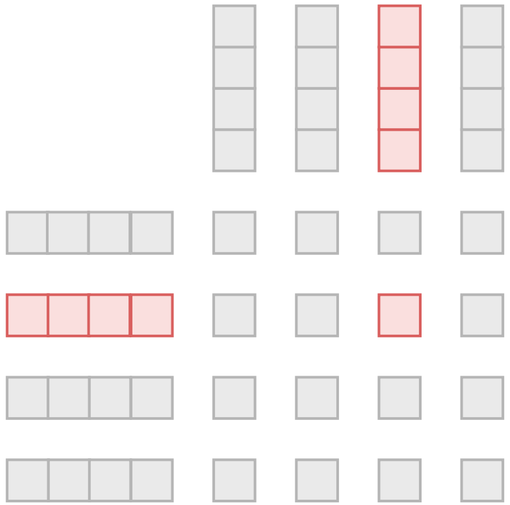


图 2.4 (a)　词与词

$$
\begin{aligned}
s_{ij} &= \boldsymbol{x}_i \cdot \boldsymbol{x}_j \\
&= \sum_{k=1}^{d} x_{ik}\, x_{jk} \\
&= \boldsymbol{x}_i \boldsymbol{x}_j^{T}
\end{aligned}
\tag{2-1}
$$

点积的定义；按分量展开，x_{ik} 为 $\boldsymbol{x}_i$ 的第 k 个分量。$\boldsymbol{x}_i$ 为 $1 \times d$ 行向量，$\boldsymbol{x}_j^T$ 为 $d \times 1$ 列向量，二者相乘得到 1×1 标量，即式 (2-1)。

$s_{ij} \in \mathbb{R}$ 为标量，s_{ij} 越大，词元 i、j 的相关性越强。完整的注意力分数还包含缩放（Scaled），以控制分数的方差。①

① 度量向量相关性的方法有多种，点积是注意力机制中的标准实现。

图源：《大语言模型：训练、精调与对齐》图 2.4；依据式 (2-1)

单个词元的注意力分数向量 $\boldsymbol{S}_i$

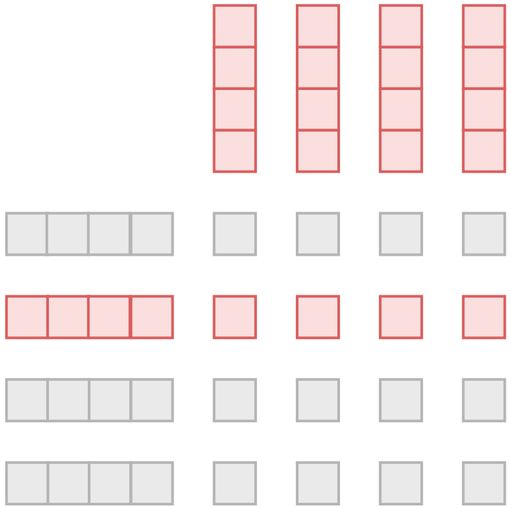


图 2.4 (b)　词与序列

$$\boldsymbol{S}_i = \boldsymbol{x}_i \boldsymbol{X}^T \tag{2-2}$$

$$\boldsymbol{X}^T = \left[\boldsymbol{x}_1^T, \cdots, \boldsymbol{x}_n^T\right]$$

$$\boldsymbol{S}_i = \boldsymbol{x}_i \left[\boldsymbol{x}_1^T, \cdots, \boldsymbol{x}_n^T\right] = \left[\boldsymbol{x}_i \boldsymbol{x}_1^T, \cdots, \boldsymbol{x}_i \boldsymbol{x}_n^T\right]$$

$$= \left[\boldsymbol{s}_{i1}, \cdots, \boldsymbol{s}_{in}\right]$$

式 (2-2) 给出 $\boldsymbol{S}_i$；$\boldsymbol{X}$ 按行堆叠，转置后 $\boldsymbol{X}^T$ 按列拼接。矩阵乘法按列分块后，每一项都是式 (2-1) 中的 $\boldsymbol{s}_{ij}$。维度为 $(1 \times d)(d \times n) = 1 \times n$。

图源：《大语言模型：训练、精调与对齐》图 2.4；依据式 (2-2)

完整的自注意力分数矩阵 S

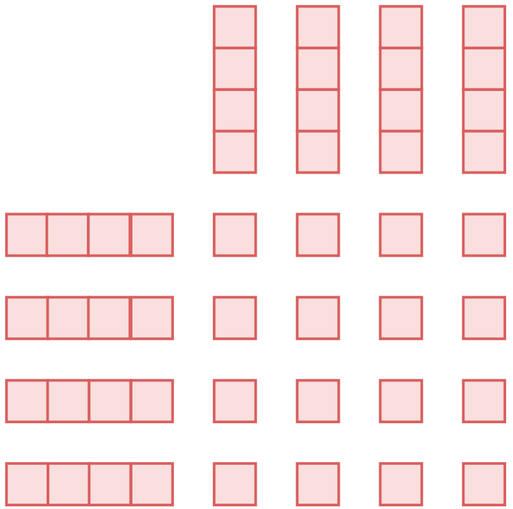


图 2.4 (c)　序列与序列

$$\boldsymbol{S} = \begin{bmatrix} \boldsymbol{S}_1 \\ \vdots \\ \boldsymbol{S}_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{x}_1 \boldsymbol{X}^T \\ \vdots \\ \boldsymbol{x}_n \boldsymbol{X}^T \end{bmatrix} = \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_n \end{bmatrix} \boldsymbol{X}^T = \boldsymbol{X}\boldsymbol{X}^T \tag{2-3}$$

第一步把各 $\boldsymbol{S}_i$ 按行堆叠，并代入式 (2-2)；第二步把公共因子 $\boldsymbol{X}^T$ 提到右侧，左侧堆叠的行向量即 $\boldsymbol{X}$，得到式 (2-3)。维度为 $n \times d$ 乘 $d \times n$，得 $n \times n$。

矩阵 $\boldsymbol{S}$ 的第 i 行第 j 列元素即 s_{ij}，包含序列中所有词元对之间的自注意力分数。

图源：《大语言模型：训练、精调与对齐》图 2.4；依据式 (2-3)

多头注意力：并行计算多组注意力关系

阅读第 10 章情节时，若注意力仅与第 1 章存在强关联，就可能忽视与第 6 章等其他章节的重要联系。多头注意力（MHA）通过并行计算多个独立的注意力关系，捕获更丰富的上下文信息。

单头

只有一组注意力关系

一个分数矩阵只能呈现一种依赖模式，容易遗漏其他重要联系。

多头

多组独立的注意力关系

将词向量沿特征维切分为 h 个头，每个头各自计算注意力分数，再分别读取。

$\boldsymbol{X} \in \mathbb{R}^{n \times d}$

原词向量矩阵

→

$\boldsymbol{X}^{(1)}, \cdots, \boldsymbol{X}^{(h)}$

沿特征维切分为 h 份

→

$\boldsymbol{S}^{(1)}, \cdots, \boldsymbol{S}^{(h)}$

每个头各自的分数矩阵

多头注意力关系（$h = 2$）

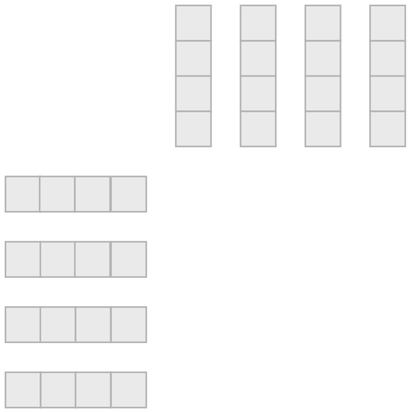


图 2.5 (a)　原始维度

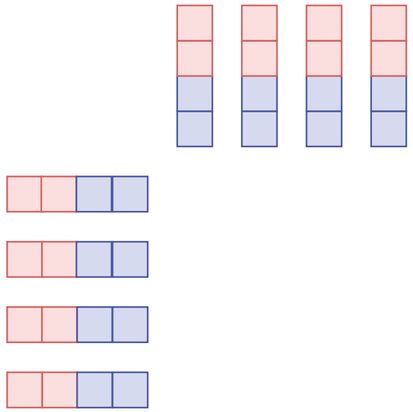


图 2.5 (b)　多头维度

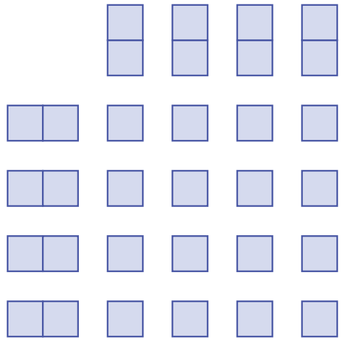


图 2.5 (c)　注意力头 1

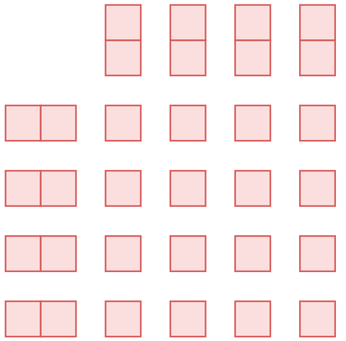


图 2.5 (d)　注意力头 2

- (a) 原始的输入序列向量表示；(b) 将每个词向量沿特征维切分为两个头（$h = 2$）。
- (c)、(d) 分别示意两个头独立计算出的注意力权重矩阵。
- 不同头关注不同依赖：一个头关注章节间的顺序依赖（第 10 章与第 1 章），另一个头关注主题联系（第 10 章与第 6 章）。

图源：《大语言模型：训练、精调与对齐》图 2.5

各头的注意力分数矩阵 $\boldsymbol{S}^{(k)}$

$$\boldsymbol{X}=\left[\boldsymbol{X}^{(1)},\boldsymbol{X}^{(2)},\cdots,\boldsymbol{X}^{(h)}\right],\quad \boldsymbol{X}^{(k)}\in\mathbb{R}^{n\times d'}$$

$$d=h\cdot d'\ \Rightarrow\ d'=d/h$$

$$\boldsymbol{S}^{(k)}=\boldsymbol{X}^{(k)}\left(\boldsymbol{X}^{(k)}\right)^{T}\tag{2-4}$$

1 · 切分

把每个词向量沿特征维切为 h 份，每份 $d'=d/h$ 维。

2 · 计算

式 (2-3) 中的 $\boldsymbol{X}$ 换成 $\boldsymbol{X}^{(k)}$，维度 $(n\times d')(d'\times n)=n\times n$。

3 · 输出

得到 h 个 $n\times n$ 的分数矩阵，各头学习不同的依赖模式。

依据式 (2-4)，参见第 2.3.4 节（多头注意力机制）

2.3

Transformer 模型结构

先看翻译数据与训练目标，再逐层拆解输入层、编码器、解码器与输出层，并看清注意力层在其中的位置。

本节内容

2.3.1 机器翻译数据

2.3.2 Transformer 模型架构和训练目标

2.3.3 输入层

2.3.4 编码器

2.3.5 解码器

2.3.6 输出层

文本先被切分为词元序列

分词器把文本转换为词元序列 $\boldsymbol{w} = (w_1, w_2, \cdots, w_n)$，每个词元都属于词表 $\mathcal{V}$。序列两端需要加入三类特殊词元。

特殊词元

<SOS>　句首

标记句子开始。解码器看到它时启动生成。

特殊词元

<EOS>　句尾

标记句子结束。解码器输出它时终止生成。

特殊词元

<PAD>　填充

把批量内长度不一的序列补齐到相同长度，便于批量训练。

这些人工定义的词元划定了变长序列的始与终，是后续生成与批量训练的基础。

双语平行语料：输入与标签

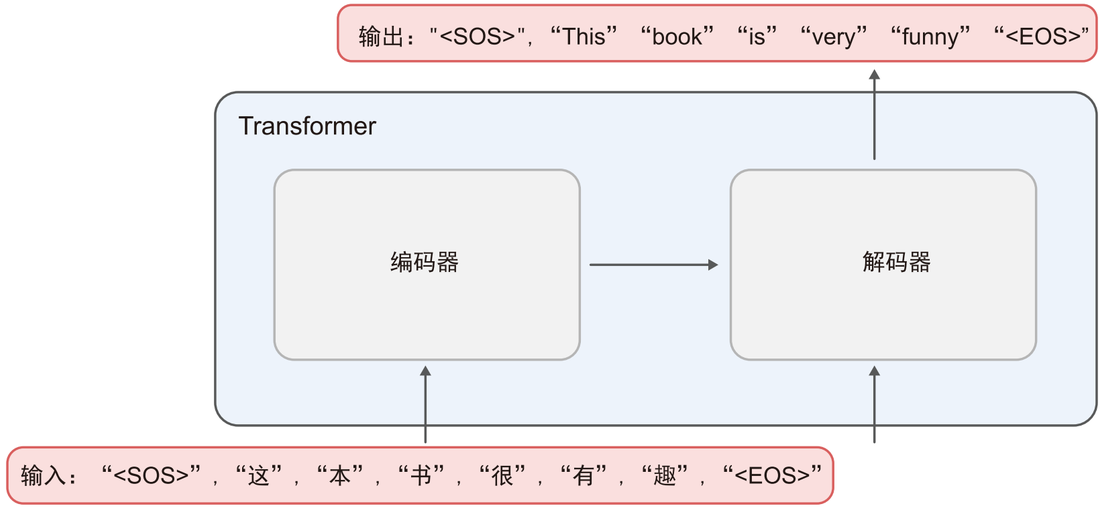


图 2.6　Transformer 模型输入输出是双语成对的数据

- 翻译数据由不同语言的平行文本对组成：源语言 $\boldsymbol{x} = (x_1, \cdots, x_m)$ 作为输入，目标语言 $\boldsymbol{y} = (y_1, \cdots, y_n)$ 作为标签。
- 两种语言使用独立的分词器与词表。图 2.6 只展示输入与输出，未包含解码器输入细节，见图 2.7。

图源 《大语言模型：训练、精调与对齐》图 2.6

编码器-解码器结构与自回归预测

早期翻译模型

编码器把输入压缩为一个上下文向量；解码器据此逐步生成译文。

Transformer

源序列 $\boldsymbol{x}$ 经编码器得到隐状态矩阵 $\boldsymbol{Z} \in \mathbb{R}^{m \times d}$；解码器利用 $\boldsymbol{Z}$ 与已生成的 $\boldsymbol{y}_{\leqslant t}$ 预测下一个词元 y_{t+1}。

$$p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right)$$

解码器是自回归模型：时刻 t 的预测依赖源序列 $\boldsymbol{x}^{(k)}$ 与已生成的前缀 $\boldsymbol{y}_{\leqslant t}^{(k)} = (y_1^{(k)}, \cdots, y_t^{(k)})$。

数据集 $\mathcal{D}$ 含 K 个样本对 $\left(\boldsymbol{x}^{(k)}, \boldsymbol{y}^{(k)}\right)$，$\boldsymbol{x}^{(k)}$ 为源语言词元序列，$\boldsymbol{y}^{(k)}$ 为目标语言词元序列。

架构：输入层、编码器、解码器与输出层

输入层

词嵌入 + 位置编码（2.3.3）

→

编码器

多层编码块（2.3.4）

→

解码器

多层解码块（2.3.5）

→

输出层

线性层 + Softmax（2.3.6）

整体共有 3 种注意力层，它们都是后面各层的核心部件：

注意力 ①

编码器自注意力

源序列内部的词元互相读取，见 2.3.4。

注意力 ②

解码器掩码自注意力

目标位置只能读取它及之前的词元，见 2.3.5。

注意力 ③

编码-解码注意力

解码器读取编码器输出 $\boldsymbol{Z}$，把源语言信息送入解码，见 2.3.5。

三类注意力层都由 2.2 节的缩放点积注意力与多头机制构成，区别只在于 $\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 的来源与所用掩码。

整体框架与三种注意力的位置

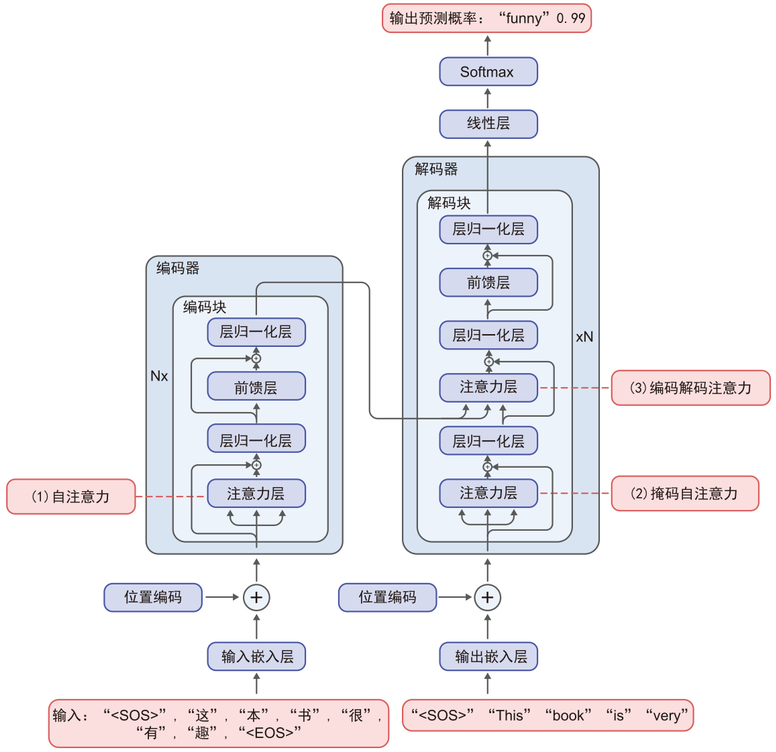


图 2.7　Transformer 网络架构图

- ①　编码器自注意力（2.3.4）：源序列内部互相读取。
- ②　解码器掩码自注意力（2.3.5）：目标序列只看已生成的部分。
- ③　编码-解码注意力（2.3.5）：目标位置读取编码器输出 $\boldsymbol{Z}$。
- 三者都由 2.2 节的缩放点积注意力与多头机制构成。

左侧为编码器堆叠，右侧为解码器堆叠；两侧之间只有编码-解码注意力的箭头，源语言信息由此进入解码。

图源　《大语言模型：训练、精调与对齐》图 2.7

训练样本：用已知前缀预测下一个词元

源句“<SOS>这本书很有趣<EOS>”输入编码器。解码器输入为目标语言前缀，监督标签为下一个词元。

<SOS> This book is very

解码器输入（已知前缀）

→

funny

预测标签（下一个词元）

- 输出层给出词表中所有词元的概率分布，而非直接给出某个词元。
- 训练目标是最大化标签 “funny” 对应的概率，即监督学习。
- 目标序列含多个词元时，每个位置都构成一个预测任务。

单个样本的损失：负对数似然

$$
\begin{aligned}
&\max_{\theta} \prod_{t=1}^{T_k} p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right) \\
&= \max_{\theta} \sum_{t=1}^{T_k} \log p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right) \\
&= \min_{\theta} \left[-\sum_{t=1}^{T_k} \log p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right)\right] \\
\mathcal{L}^{(k)}(\theta) &= -\frac{1}{T_k} \sum_{t=1}^{T_k} \left[\log p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right)\right]
\end{aligned}
$$

式 (2-5)：依次为最大化似然、取对数、取负号，再按长度 T_k 取平均。

注 T_k 为第 k 条数据的词元序列长度；$p_\theta(\cdot)$ 为参数 θ 下的预测分布。

数据集的总损失

$$
\begin{aligned}
\mathcal{L}(\theta) &= \frac{1}{K}\sum_{k=1}^{K}\mathcal{L}^{(k)}(\theta) \\
&= \frac{1}{K}\sum_{k=1}^{K}\left(-\frac{1}{T_k}\sum_{t=1}^{T_k}\log p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right)\right) \\
&= -\frac{1}{K}\sum_{k=1}^{K}\frac{1}{T_k}\sum_{t=1}^{T_k}\log\left(p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right)\right)
\end{aligned}
\tag{2-6}
$$

第一步对 K 个样本级损失取平均；第二步代入式 (2-5)；第三步去掉外层括号，即式 (2-6)。

最小化 $\mathcal{L}(\theta)$ 等价于最大化预测目标序列的概率；它是标准交叉熵损失在自回归序列建模中的形式。

输入层：词嵌入与位置编码

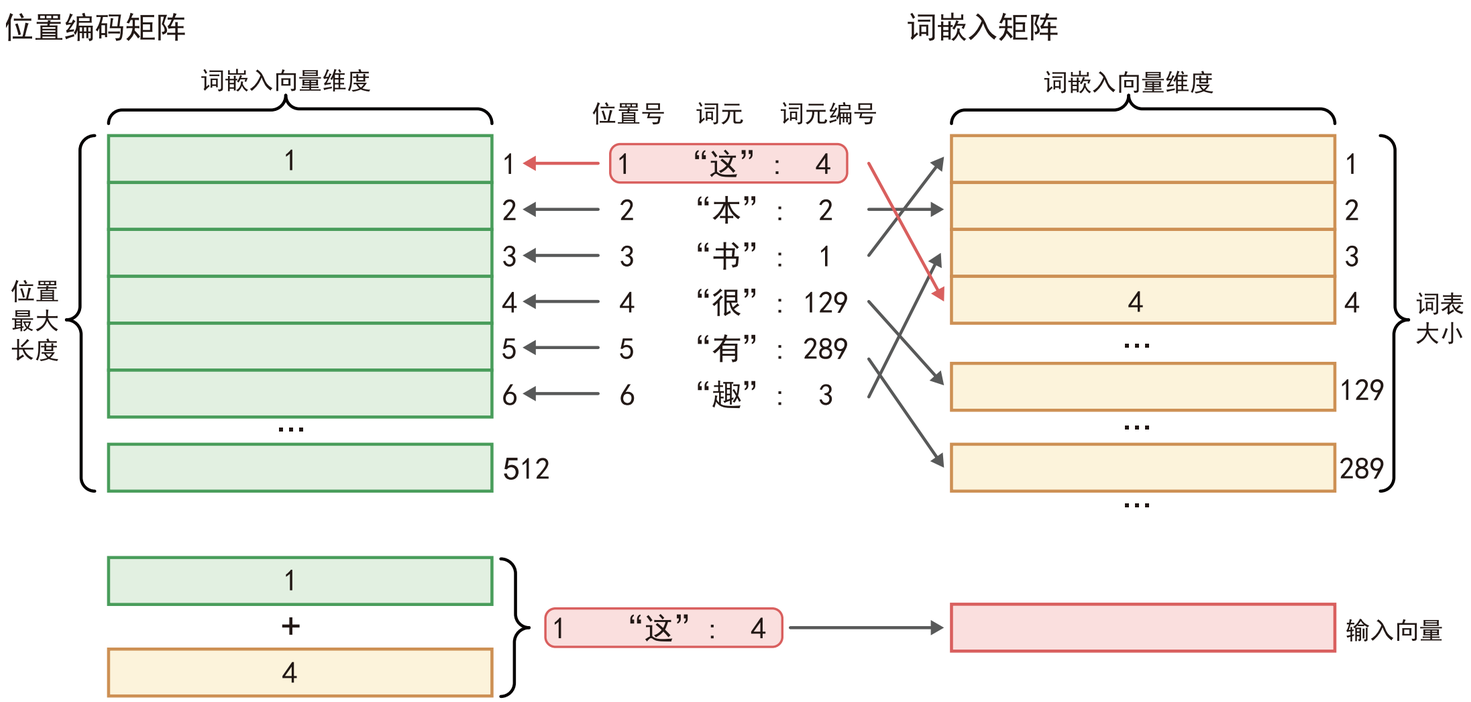


图 2.8　Transformer输入层示例

- 以首位词元“这”（词元编号 4）为例。
- 嵌入层从嵌入参数矩阵中查得它的词嵌入向量。
- 位置编码层从位置编码矩阵中查得该位置的向量。
- 两者相加，得到融合位置信息的词向量。

图源 《大语言模型：训练、精调与对齐》图 2.8

嵌入层：按词元编号查表

$$\boldsymbol{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$$

可学习的词嵌入参数矩阵，每一行对应词表中的一个词元。

$$\boldsymbol{x}_i = \boldsymbol{E}_{x_i} \in \mathbb{R}^{1 \times d}$$

取出第 x_i 行，即词元编号 x_i 对应的词嵌入向量。

$$\boldsymbol{X} = \left[\boldsymbol{x}_1; \boldsymbol{x}_2; \cdots; \boldsymbol{x}_n\right] \in \mathbb{R}^{n \times d}$$

按行拼接得到输入的词嵌入矩阵。

图 2.9 中 6 个词元即被表示为 $\boldsymbol{X} \in \mathbb{R}^{6 \times d}$。

嵌入矩阵的查表过程与初始化

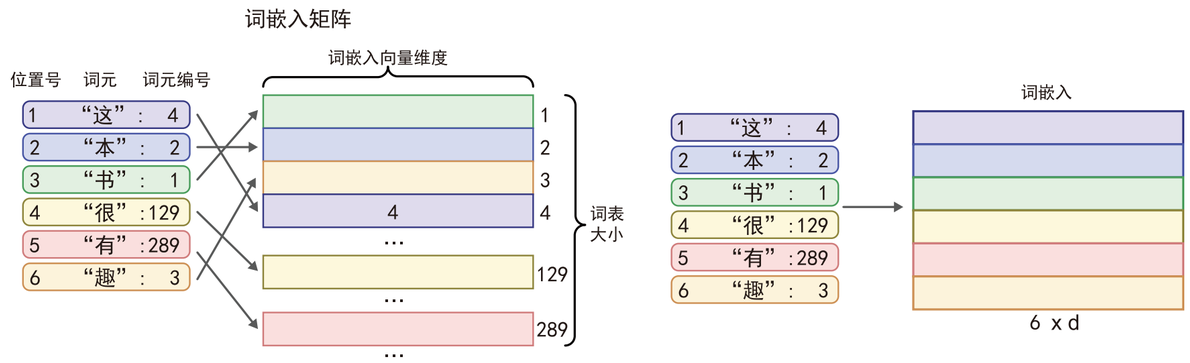


图 2.9　词元编号序列在嵌入层参数矩阵逐行取向量组成词嵌入输入矩阵

初始化

正态分布

$\boldsymbol{E}$ 通常用均值为 0、标准差 $\sigma = 0.02$ 的正态分布初始化，并随训练通过反向传播学习。

填充符

<PAD> 为零向量

填充符只用于统一长度，无实际语义，其词嵌入通常设为零向量。

独立嵌入

编码器与解码器分开

两侧处理不同语言、拥有各自的词表，因此使用独立的输入嵌入层。

图源 《大语言模型：训练、精调与对齐》图 2.9

为什么必须显式加入位置信息

语序的改变可能改变语义，模型需要同时感知词元的绝对位置与词元之间的相对位置。

问题

Transformer 不按顺序递归

RNN 类模型按时间步递归处理序列，天然携带顺序；Transformer 不能这样做，需要把位置信息显式加入词嵌入。

约束

自注意力对顺序不敏感

位置信息弥补了注意力机制本身对序列顺序不敏感的缺陷，使模型能理解序列特性。

要求 1

唯一性

每个位置的编码向量都不同。

要求 2

稠密性

所有维度都包含有效信息，避免固定为 0 的冗余维度。

要求 3

多频率

不同维度捕获不同变化频率的位置模式。

二进制位置编码：唯一，但维度利用率低

表 2.1　二进制位置编码向量

位置号	β 转换	二进制编码向量
0	β_0	[0,0,0,0,0,0,0,0]
1	β_1	[0,0,0,0,0,0,0,1]
2	β_2	[0,0,0,0,0,0,1,0]
3	β_3	[0,0,0,0,0,0,1,1]
4	β_4	[0,0,0,0,0,1,0,0]
5	β_5	[0,0,0,0,0,1,0,1]
6	β_6	[0,0,0,0,0,1,1,0]
7	β_7	[0,0,0,0,0,1,1,1]
⋮	⋮	⋮
15	β_{15}	[0,0,0,0,1,1,1,1]
64	β_{64}	[0,1,0,0,0,0,0,0]

- 位置编码矩阵 $\mathbf{PE} \in \mathbb{R}^{L_{\max} \times d}$，通常预先算好、训练时不学习。
- 每个位置号对应唯一的 8 位编码。
- 最右端为最低位，变化最快、周期最短；最左端为最高位，变化最慢、周期最长。
- 当 $d = 512$、$L_{\max} = 1024$ 时，仅需低 $\lceil \log_2 1024 \rceil = 10$ 位即可唯一表示所有位置，高 502 位始终为 0，成为冗余维度，模型难以利用。

正弦位置编码（式 (2-7)）

$$\begin{aligned}\mathbf{PE}_m &= \left[\sin(m\theta_1), \cos(m\theta_1), \sin(m\theta_2), \cos(m\theta_2), \cdots, \sin(m\theta_t), \cos(m\theta_t)\right] \\ &= \left[\boldsymbol{p}_{m,1}, \boldsymbol{p}_{m,2}, \cdots, \boldsymbol{p}_{m,t}\right], \qquad \boldsymbol{p}_{m,i} = \left(\sin(m\theta_i),\ \cos(m\theta_i)\right)\end{aligned}$$

式 (2-7)：每两维为一组，第 i 组由同一角度 $m\theta_i$ 的正弦与余弦组成，再按组拼接。

$$\theta_i = \frac{1}{b^{2(i-1)/d}}$$

m

位置号

词元在序列中的位置，从 0 开始。

$t = d/2$

角度组数

d 维词向量每两维一组，共 t 组。

θ_i

第 i 组角度

$\theta_1 = 1$，随 i 增大而减小。

b

基数 $b = 10\,000$

使 θ_i 落在合适的取值范围内。

角度随维度衰减：由高频到低频

$\theta_1 = \dfrac{1}{b^0} = 1$，$\theta_t = \dfrac{1}{b^{(d-2)/d}}$，即 $\dfrac{1}{b^{(d-2)/d}} \leqslant \theta_i \leqslant 1$。随 i 增大，$\sin(m\theta_i)$ 与 $\cos(m\theta_i)$ 的频率降低，高维变化更平缓。

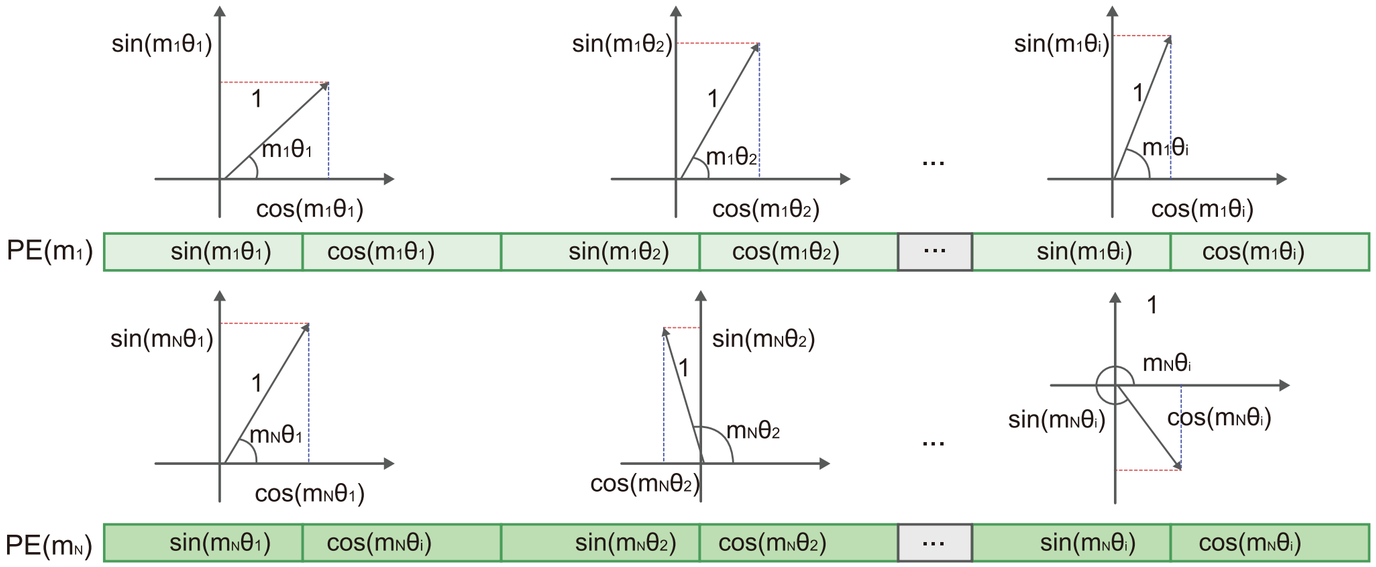


图 2.10　位置编码向量由不同角度的正弦和余弦构成

图源 《大语言模型：训练、精调与对齐》图 2.10

输入表示：词嵌入与位置编码相加（式 (2-8)）

$$\boldsymbol{x}_i' = \boldsymbol{x}_i + \mathrm{PE}_i$$

式 (2-8)：$\boldsymbol{x}_i$ 与 PE_i 同为 $1 \times d$，逐元素相加后仍为 $1 \times d$。

$$\boldsymbol{X}' = [\boldsymbol{x}_1 + \mathrm{PE}_1;\ \cdots;\ \boldsymbol{x}_n + \mathrm{PE}_n] = \boldsymbol{X} + [\mathrm{PE}_1;\ \cdots;\ \mathrm{PE}_n] \in \mathbb{R}^{n \times d}$$

按行堆叠得到输入矩阵，形状保持 $n \times d$。

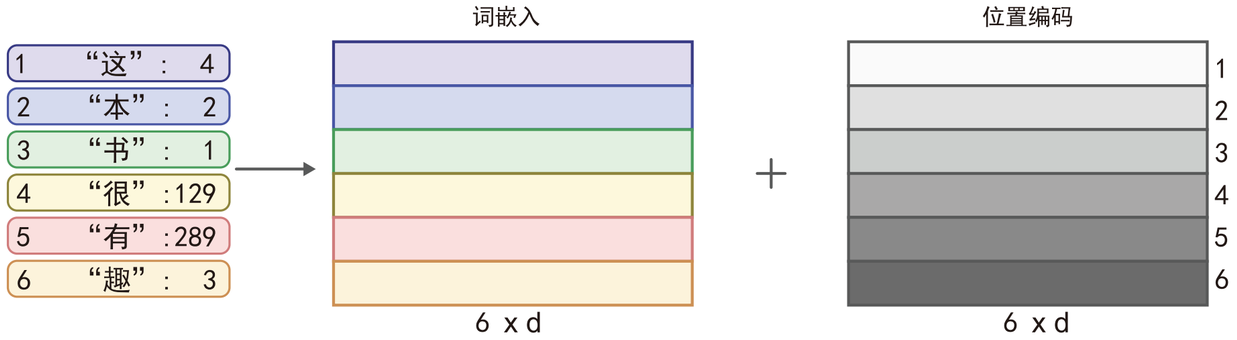


图 2.11　Transformer 输入层处理示例

图源 《大语言模型：训练、精调与对齐》图 2.11

位置信息的融合方式与 Dropout

方式	做法
逐元素操作	每个元素乘以位置相关的标量（缩放）或加上位置相关的偏置（平移）
向量加法（最常用）	词嵌入与位置编码相加，即式 (2-8)；可视为引入位置相关的偏置，能加速收敛
矩阵变换	位置编码通过矩阵乘法作用于词向量，如旋转位置编码 RoPE（详见第 4 章）
向量拼接	位置编码与词嵌入拼接，但会增加特征维度
学习式编码	用可学习参数向量表示位置信息

训练时

输入层的 Dropout

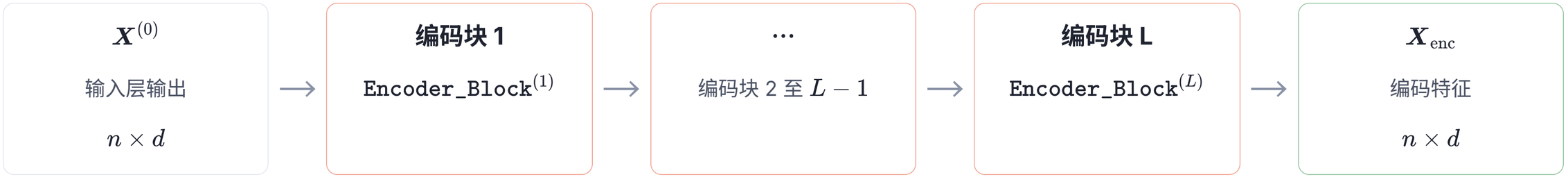
为提升鲁棒性，可对 $\boldsymbol{x}_i'$ 以概率 p 随机将部分维度置零。随着特征维度增加，当前主流大模型往往省略输入层的 Dropout。

编码器：多个编码块堆叠

编码器对输入词元序列逐块变换，最终为每个词元输出编码特征 $\boldsymbol{X}_{\text{enc}}$，供解码器的各个解码块读取。

$$\boldsymbol{X}^{(l)} = \texttt{Encoder_Block}^{(l)}\left(\boldsymbol{X}^{(l-1)}\right) \tag{2-9}$$

$\boldsymbol{X}^{(l)}$ 为第 l 个编码块的输出，维度与输入 $\boldsymbol{X}^{(l-1)}$ 一致；$\boldsymbol{X}^{(0)}$ 为输入层的输出，即首个编码块的输入。



- 每个编码块的输入与输出维度相同，因此块可以一层一层堆叠。
- $\boldsymbol{X}_{\text{enc}} \in \mathbb{R}^{n \times d}$ 会被送入解码器的每个解码块（见 2.3.5）。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-9)）

编码块的组成

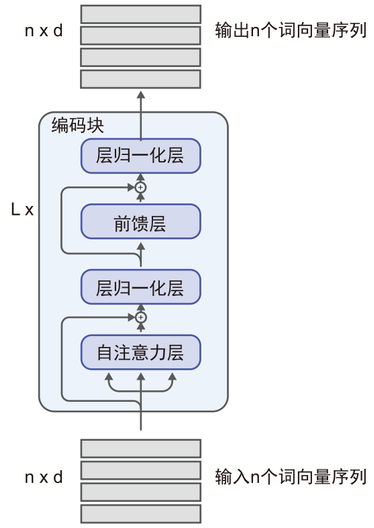


图 2.12　编码块的输入输出维度一致

残差连接：子层输出与子层输入相加

子层 1

自注意力层

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 均来自本块输入；掩码 $\boldsymbol{M}$ 屏蔽 <PAD> 等不参与计算的词元。

归一化

层归一化（LN）

`Layer_Norm_1` 在自注意力层之后，`Layer_Norm_2` 在前馈层之后。

子层 2

前馈层

先升维做非线性激活，再降维还原为 d 维。

图 《大语言模型：训练、精调与对齐》图 2.12

编码块的计算：两个子层，各接残差

$$\boldsymbol{X}_{\text{attn}} = \texttt{Multi_Head_Attention}\left(\boldsymbol{X}^{(l-1)}, \boldsymbol{M}\right) \tag{2-10}$$

$$\boldsymbol{X}_{\text{res1}} = \boldsymbol{X}_{\text{attn}} + \boldsymbol{X}^{(l-1)} \tag{2-11}$$

$$\boldsymbol{X}_{\text{ln1}} = \texttt{Layer_Norm_1}\left(\boldsymbol{X}_{\text{res1}}\right) \tag{2-12}$$

$$\boldsymbol{X}_{\text{ffn}} = \texttt{Feed_Forward}\left(\boldsymbol{X}_{\text{ln1}}\right) \tag{2-13}$$

$$\boldsymbol{X}_{\text{res2}} = \boldsymbol{X}_{\text{ffn}} + \boldsymbol{X}_{\text{ln1}} \tag{2-14}$$

$$\boldsymbol{X}^{(l)} = \texttt{Layer_Norm_2}\left(\boldsymbol{X}_{\text{res2}}\right) \tag{2-15}$$

式 (2-10)–(2-12) 为注意力子层，式 (2-13)–(2-15) 为前馈子层。每个子层“先残差相加，再归一化”：$\boldsymbol{X}_{\text{res1}}$、$\boldsymbol{X}_{\text{res2}}$ 是残差相加后的中间结果。

注意力子层

$\boldsymbol{X}^{(l-1)}$ 经多头注意力得到 $\boldsymbol{X}_{\text{attn}}$，与 $\boldsymbol{X}^{(l-1)}$ 相加后再归一化，得到 $\boldsymbol{X}_{\text{ln1}}$。

前馈子层

$\boldsymbol{X}_{\text{ln1}}$ 经前馈层得到 $\boldsymbol{X}_{\text{ffn}}$，与 $\boldsymbol{X}_{\text{ln1}}$ 相加后再归一化，得到 $\boldsymbol{X}^{(l)}$；维度始终不变。

顺序 残差相加在归一化之前：主干路径上保留一条不做归一化的直通支路，梯度可以无衰减地回传，这正是深层堆叠能训练起来的原因。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-10)–(2-15)）

注意力：对词元序列做加权组合

注意力是序列特征的组合机制：每个词元的新表示，由序列中各词元的表示加权得到，能够捕捉长距离依赖。下面的分数定义与 2.2.3 节一致。

$$\boldsymbol{X}' = \texttt{Multi_Head_Attention}(\boldsymbol{X}, \boldsymbol{M}) \tag{2-16}$$

$\boldsymbol{X}, \boldsymbol{X}' \in \mathbb{R}^{n\times d}$，$n$ 为词元序列长度，d 为特征维度；$\boldsymbol{M}$ 为掩码矩阵。

$$s_{ij} = \boldsymbol{x}_i \boldsymbol{x}_j^T \tag{2-17}$$

任意两个词元特征 $\boldsymbol{x}_i, \boldsymbol{x}_j \in \mathbb{R}^{1\times d}$ 之间的注意力分数，度量词元 i 与词元 j 的相关程度，决定 $\boldsymbol{x}_j$ 对 $\boldsymbol{x}_i$ 的贡献。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-16)、(2-17)）

投影：Q、K、V 来自同一输入

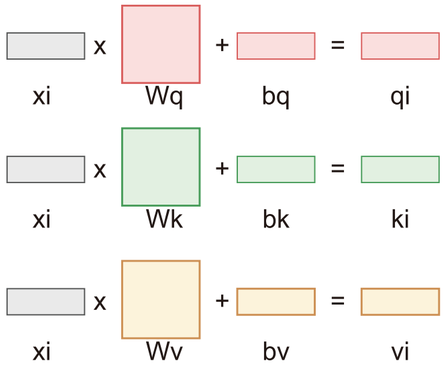


图 2.13(a)　投影变换

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_q + \boldsymbol{b}_q \tag{2-18}$$

$$\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_k + \boldsymbol{b}_k \tag{2-19}$$

$$\boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_v + \boldsymbol{b}_v \tag{2-20}$$

$\boldsymbol{X} \in \mathbb{R}^{n\times d}$ 每行是一个词向量；$\boldsymbol{W}_q, \boldsymbol{W}_k, \boldsymbol{W}_v \in \mathbb{R}^{d\times d}$ 为可学习的投影矩阵，$\boldsymbol{b}_q, \boldsymbol{b}_k, \boldsymbol{b}_v$ 加到每一行上。

- **自注意力**中，$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 都由同一个输入 $\boldsymbol{X}$ 投影得到。
- $\boldsymbol{Q}$、$\boldsymbol{K}$ 主要用于计算相关性，$\boldsymbol{V}$ 用于特征学习。
- 训练时投影矩阵可学习，从而改变词元之间的注意力分数。

图 《大语言模型：训练、精调与对齐》图 2.13(a)；式 (2-18)–(2-20)

注意力分数：查询与键的点积并缩放

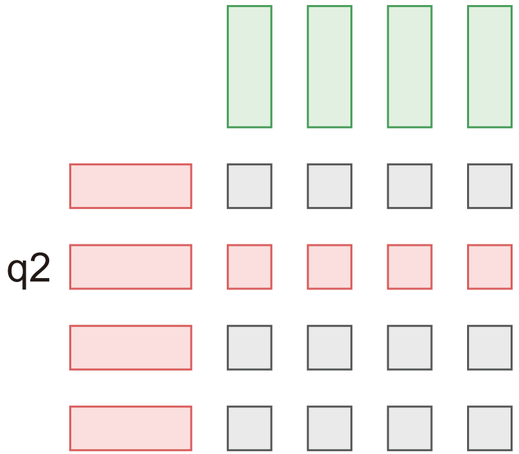


图 2.13(b)　注意力权重计算

$$\boldsymbol{S}_i = \boldsymbol{q}_i \boldsymbol{K}^T \tag{2-21}$$

第 i 个查询与所有键的分数行向量，$\boldsymbol{S}_i \in \mathbb{R}^{1\times n_k}$。

$$\boldsymbol{S} = \frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d}} \tag{2-22}$$

$\boldsymbol{S} \in \mathbb{R}^{n_q \times n_k}$；自注意力中 $n_q = n_k = n$；除以 $\sqrt{d}$ 控制分数的尺度。

- $\boldsymbol{S}$ 的第 i 行对应查询词元 i，第 j 列对应键词元 j。

图 《大语言模型：训练、精调与对齐》图 2.13(b)；式 (2-21)、(2-22)

为什么要除以 $\sqrt{d}$

$$\boldsymbol{q}\cdot\boldsymbol{k}=\sum_{m=1}^{d}q_m k_m$$

$$\mathbb{E}\left[q_m k_m\right]=\mathbb{E}\left[q_m\right]\mathbb{E}\left[k_m\right]=0$$

$$\operatorname{Var}\left(q_m k_m\right)=\mathbb{E}\left[\left(q_m k_m\right)^2\right]=\mathbb{E}\left[q_m^2\right]\mathbb{E}\left[k_m^2\right]=1$$

$$\operatorname{Var}\left(\boldsymbol{q}\cdot\boldsymbol{k}\right)=\sum_{m=1}^{d}\operatorname{Var}\left(q_m k_m\right)=d$$

$$\operatorname{Var}\left(\frac{\boldsymbol{q}\cdot\boldsymbol{k}}{\sqrt{d}}\right)=\frac{1}{d}\operatorname{Var}\left(\boldsymbol{q}\cdot\boldsymbol{k}\right)=1$$

假设 $\boldsymbol{q}$、$\boldsymbol{k}$ 的各分量相互独立，且均服从均值为 0、方差为 1 的标准正态分布。

不缩放

点积的标准差为 $\sqrt{d}$，维度越大，$|\boldsymbol{q}\cdot\boldsymbol{k}|$ 越大。

缩放后

除以 $\sqrt{d}$ 后方差回到 1，Softmax 的输入保持在梯度有效的范围内。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-22) 的缩放说明）

注意力权重与加权求和

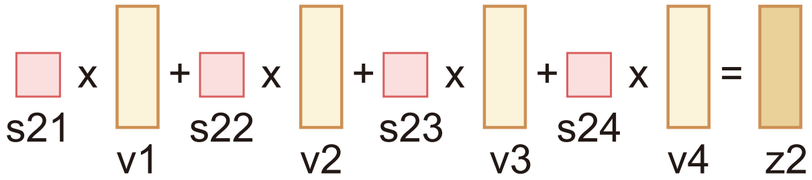


图 2.13(c)　特征加权组合

$$\boldsymbol{P}_i = \mathtt{Softmax}\left(\boldsymbol{S}_i\right) \tag{2-23}$$

$$\boldsymbol{z}_i = \sum_{j=1}^{n_k} P_{ij}\boldsymbol{v}_j = \boldsymbol{P}_i\boldsymbol{V} \tag{2-24}$$

$$\boldsymbol{Z} = \boldsymbol{P}\boldsymbol{V} \tag{2-25}$$

逐行 Softmax：$P_{ij} = \dfrac{\exp(S_{ij})}{\sum_k \exp(S_{ik})}$，每行之和为 1。

- 每个查询 i 都有一份独立的权重分布。
- $\boldsymbol{z}_i$ 是 $\boldsymbol{x}_i$ 融合上下文后的新表示；$\boldsymbol{Z} \in \mathbb{R}^{n_q \times d}$。

图 《大语言模型：训练、精调与对齐》图 2.13(c)；式 (2-23)–(2-25)

缩放点积注意力的完整流程



$$\boldsymbol{O} = \boldsymbol{Z}\boldsymbol{W}_o + \boldsymbol{b}_o \tag{2-26}$$

$\boldsymbol{W}_o \in \mathbb{R}^{d\times d}$、$\boldsymbol{b}_o \in \mathbb{R}^{d}$ 为可学习的线性投影，进一步增强表示能力；$\boldsymbol{O} \in \mathbb{R}^{n_q \times d}$。

合并上面各步，即 $\boldsymbol{Z} = \mathtt{Softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d}}\right)\boldsymbol{V}$。这就是本章 2.2 节给出的缩放点积注意力。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-18)–(2-26)）

数值示例：三个词元从分数矩阵到输出向量

给定序列（大、模、型），经输入层得到 $\boldsymbol{X} \in \mathbb{R}^{3\times d}$，再投影为 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$，逐元素算得分数 $s_{ij} = \boldsymbol{q}_i \boldsymbol{k}_j^T / \sqrt{d}$（式 (2-27)），即表 2.2；再按行做 Softmax、加权求和得到输出。

表 2.2　注意力计算实例（行对应查询，列对应键）

$\boldsymbol{q}_i \boldsymbol{k}_j^T / \sqrt{d}$	$\boldsymbol{k}_{大}$	$\boldsymbol{k}_{模}$	$\boldsymbol{k}_{型}$
$\boldsymbol{q}_{大}$	$s_{大,大}$	$s_{大,模}$	$s_{大,型}$
$\boldsymbol{q}_{模}$	$s_{模,大}$	$s_{模,模}$	$s_{模,型}$
$\boldsymbol{q}_{型}$	$s_{型,大}$	$s_{型,模}$	$s_{型,型}$

- 每个查询 $\boldsymbol{q}_i$ 与全部键 $\boldsymbol{k}_j$ 算一行分数，三个词元得到 3×3 的分数矩阵。
- 表中是 Softmax 之前的分数 s_{ij}。

$$\boldsymbol{P}_i = \texttt{Softmax}\left(\boldsymbol{S}_i\right) \tag{2-28}$$

$$\boldsymbol{P}_{大} = \texttt{Softmax}\left(\boldsymbol{S}_{大}\right) \tag{2-29}$$

$$\boldsymbol{z}_i = \boldsymbol{P}_i \boldsymbol{V} = \sum_j P_{ij} \boldsymbol{v}_j \tag{2-30}$$

$$\begin{aligned} \boldsymbol{z}_{大} = \boldsymbol{P}_{大} \boldsymbol{V} &= P_{大,大} \boldsymbol{v}_{大} + P_{大,模} \boldsymbol{v}_{模} \\ &\quad + P_{大,型} \boldsymbol{v}_{型} \end{aligned} \tag{2-31}$$

$$\boldsymbol{Z} = [\boldsymbol{z}_1; \boldsymbol{z}_2; \boldsymbol{z}_3] = [\boldsymbol{z}_{大}; \boldsymbol{z}_{模}; \boldsymbol{z}_{型}] \tag{2-32}$$

$\boldsymbol{Z} \in \mathbb{R}^{3\times d}$，与输入 $\boldsymbol{X}$ 的维度一致。$P_{大,大} + P_{大,模} + P_{大,型} = 1$，“大”的关注权重合计为 1。

多头注意力：并行的多个注意力头

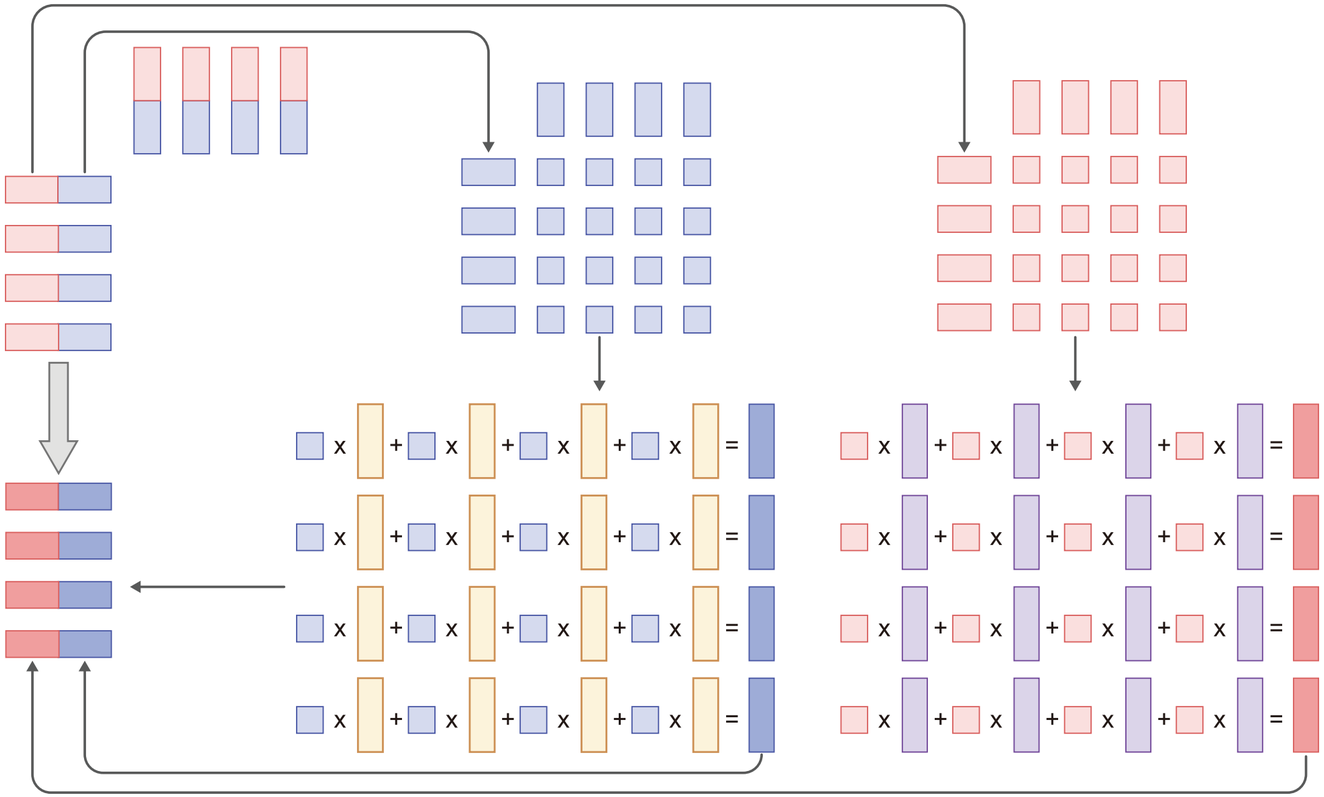


图 2.14　多头注意力机制（图示为两个注意力头）

- 单一注意力头的表示能力有限，上下文表示可能不够丰富。
- 沿特征维度切分后，各头在不同的表示子空间中学习不同的注意力模式。
- 各头彼此独立，可以并行计算，显著提高效率。

两个头各自计算一份 $\boldsymbol{S}$ 与 $\boldsymbol{Z}$，最后沿特征维拼接，再由 $\boldsymbol{W}_o$ 融合（式 (2-36)–(2-38)）。

沿特征维度切分 Q、K、V

$$\boldsymbol{Q} \to \left[\boldsymbol{Q}^{(1)}, \boldsymbol{Q}^{(2)}, \cdots, \boldsymbol{Q}^{(h)}\right], \quad \boldsymbol{Q}^{(i)} \in \mathbb{R}^{n \times d'} \tag{2-33}$$

$$\boldsymbol{K} \to \left[\boldsymbol{K}^{(1)}, \boldsymbol{K}^{(2)}, \cdots, \boldsymbol{K}^{(h)}\right], \quad \boldsymbol{K}^{(i)} \in \mathbb{R}^{n \times d'} \tag{2-34}$$

$$\boldsymbol{V} \to \left[\boldsymbol{V}^{(1)}, \boldsymbol{V}^{(2)}, \cdots, \boldsymbol{V}^{(h)}\right], \quad \boldsymbol{V}^{(i)} \in \mathbb{R}^{n \times d'} \tag{2-35}$$

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{n \times d}$ 沿特征维度均匀切分为 h 份，$d' = d/h$，要求 d 可被 h 整除。

- 切分前后词元数 n 不变，只有特征维度被分开：每个头处理 d' 维子空间。
- 例如 $d = 512$、$h = 8$ 时，每个头的维度为 $d' = 64$。

每个头独立计算，再拼接与投影

切分
$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \to h$ 份

→

并行 SDPA
每头 d' 维 (2-36)

→

拼接
沿特征维度 (2-37)

→

投影
$\boldsymbol{W}_o$ 融合 (2-38)

$$\boldsymbol{Z}^{(i)} = \texttt{Softmax}\left(\frac{\boldsymbol{Q}^{(i)}{\boldsymbol{K}^{(i)}}^T}{\sqrt{d'}}\right)\boldsymbol{V}^{(i)} \in \mathbb{R}^{n\times d'} \tag{2-36}$$

$$\boldsymbol{Z} \leftarrow \left[\boldsymbol{Z}^{(1)}, \boldsymbol{Z}^{(2)}, \cdots, \boldsymbol{Z}^{(h)}\right] \in \mathbb{R}^{n\times d} \tag{2-37}$$

$$\boldsymbol{O} = \boldsymbol{Z}\boldsymbol{W}_o + \boldsymbol{b}_o \in \mathbb{R}^{n\times d} \tag{2-38}$$

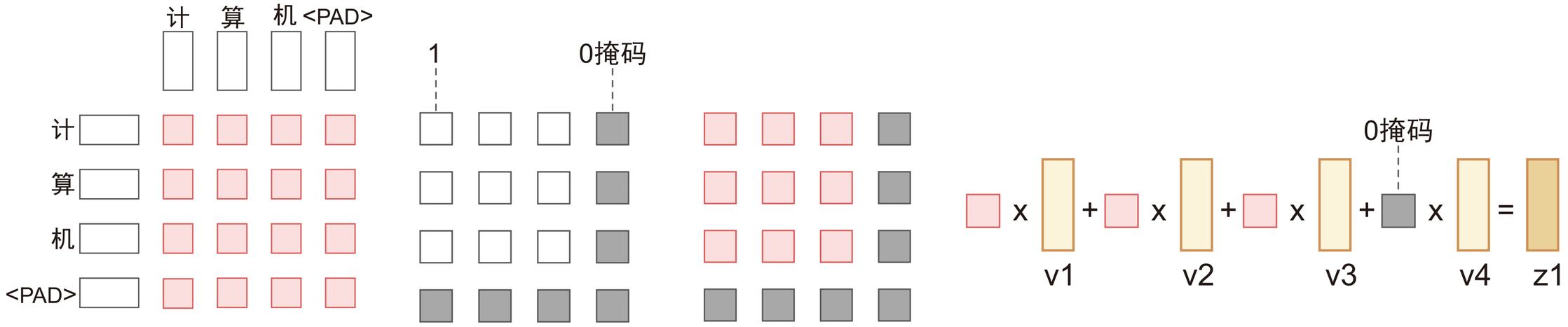
第 i 个头的缩放因子为 $\sqrt{d'}$（而非 $\sqrt{d}$）；拼接后的 $\boldsymbol{Z}$ 与输入维度相同。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-36)–(2-38)）

自注意力的掩码：屏蔽 <PAD>

$$M_{ij}^{\text{enc}} = \begin{cases} 1 & \text{if } (w_i \neq \text{<PAD>} \text{ and } w_j \neq \text{<PAD>}) \\ 0 & \text{otherwise} \end{cases} \tag{2-39}$$

$\boldsymbol{M}^{\text{enc}} \in \{0,1\}^{n_q \times n_k}$：$n_q$ 为查询词元数，n_k 为键词元数，自注意力中二者都等于序列长度 n。$M_{ij}^{\text{enc}} = 1$ 表示第 i 个查询可以读取第 j 个键，0 表示屏蔽。w_i 是第 i 个查询位置的词元，w_j 是第 j 个键位置的词元；任一侧为填充符 <PAD>，该位置即被屏蔽。



(a) 注意力分数　(b) 掩码矩阵　(c) 掩码分数　(d) 带掩码的注意力特征

图 2.15　掩码注意力机制：先由式 (2-22) 得到分数矩阵 (a)，再用掩码矩阵 (b) 把 <PAD> 对应的分数置为 $-\infty$ 得 (c)，Softmax 后这些位置的权重为 0，加权求和的输出见 (d)。

掩码的作用：被屏蔽的分数取 $-\infty$

$$S_{ij} = \begin{cases} S_{ij} & \text{if } M^{\text{enc}}_{ij} = 1 \\ -\infty & \text{if } M^{\text{enc}}_{ij} = 0 \end{cases} \tag{2-40}$$

先由式 (2-22) 得到原始分数 $\boldsymbol{S}$，再按掩码矩阵 $\boldsymbol{M}^{\text{enc}}$ 逐元素处理。

$$M^{\text{enc}}_{ij} = 0 \;\Rightarrow\; S_{ij} = -\infty \;\Rightarrow\; \exp(S_{ij}) = 0$$

$$P_{ij} = \frac{\exp(S_{ij})}{\sum_k \exp(S_{ik})} = \frac{0}{\sum_k \exp(S_{ik})} = 0$$

只要该行至少有一个未被屏蔽的分数，分母就不为零，被屏蔽位置的权重即为 0。

- 实现中通常不删除 <PAD> 对应的行与列，以保持权重与输出的维度一致，简化计算。
- 掩码只改变“哪些位置参与计算”，三类注意力的核心计算形式相同（见 2.3.5）。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-40)）

思考题与实践题

1 自注意力的时间与空间复杂度各是多少？当序列长度从 n 翻倍到 $2n$ 时，两者分别变成原来的几倍？

2 表 2.3 中 s_{ij}（$i < j$）为何置为 $-\infty$？这对 $\boldsymbol{q}_i$ 有什么影响？

3 如何把 Transformer 去掉编码器，直接用于双语翻译任务的学习？解码器的输入与注意力层要相应做哪些改动？

4 生成 m 个词元需要几次解码器前向计算？与训练相比，慢在哪里？

5 掩码自注意力里，查询 $\boldsymbol{q}_i$ 究竟在检索什么？为什么它的检索范围必须截断到位置 i？

6 位置编码点积只依赖 $j - i$：相对距离越大，注意力分数会呈现什么特点？这与“远距离依赖更难学”是什么关系？

7 实践：用第 2.5 节的代码实现一个双语翻译 Transformer，跑通“编码器 + 解码器 + 输出层”的完整前向与训练。

8 实践：推导层归一化的梯度（对 $\boldsymbol{x}_i$、$\boldsymbol{\gamma}$、$\boldsymbol{\beta}$），并实现一个可反向传播的 LayerNorm 类。